CAMARINES SUR POLYTECHNIC COLLEGES
COLLEGE of COMPUTER STUDIES

CSAC 212 - Introduction to Artificial Intelligence

Detecting Cheating in Online Chess: A Hybrid Approach Using CNN-LSTM and XGBoost for Identifying Computer Assistance

Jave Atanacio Bacsain
College of Computer Studies
Camarines Sur Polytechnic Colleges
javbacsain@my.cspc.edu.ph

Abstract

With the further development of new chess engine technologies and their growing popularity, there were more chances for cheating through computer assistance that appeared in the past. To this end, the project focused its attention on creating a cheat detection system that flagged suspected cheaters to ensure fair play on several online chess platforms. This study was a hybrid approach using Convolutional Neural Networks (CNN) along with Long Short-Term Memory (LSTM) networks to analyze chess moves. The CNN assessed the positions of the pieces on the chessboard, while the LSTM tracked the patterns of the sequences of moves made by each player. Then, an Extreme Gradient Boosting (XGBoost) algorithm was utilized to evaluate the overall game data and help determine whether cheating occurred. The dataset consisted of thousands of chess games, including information about the game's result ('1-0', '0-1', or '1/2-1/2') and cheating status (0 for not cheating, 1 for cheating) for both players, along with their Elo ratings derived from the first ten moves.

1 Introduction

The game of chess has existed for well over a thousand years, and up until the past century, the emergence of chess computers has emerged. These chess engines have improved so much in recent years that they are unbeatable by the best human players in the world [11] [18] [29]. Moreover, through the advent of the Internet age, online chess play has become prominent, which, in turn, also has contributed to the increase of cheaters through chess computer assistance [1] [7] [30]. In addition to this, there is only a limited amount of publicly available data for accessing cheating detection methods, making it crucial to have studies related to this topic [26].

Cheating does not only negatively affect the players' experiences, but also it could lead to the distortion of tournament results where rightful winners would miss out on prizes/awards. Anyone can

just cheat in any online chess game with ease by having a large amount of easily accessible strong chess computers such as the free and open-source Stockfish chess engine [25]. The integrity of online chess games could easily be compromised with just a few clicks. With this, it may lead to diminished opportunities for everyone involved, deterring even new players from entering the chess community [1] [16] [31].

The input for the program contained a dataset that comprised 48,000 chess games, with each move having a certain probability of being made by either of the chess engines Stockfish or Maia. According to BrieucDandoy [26], the author of the dataset, to save computing time, one side was given zero probability of using Stockfish for most of the game, which meant that only one side was "cheating." Each game from the dataset contained the Portable Game Notation (PGN) format of the entire match, the Elo rating or skill level of the players, the results of the game ('1-0', '0-1', or '1/2-1/2'), and the moves when the "cheat" move had been made (0 for not cheating, 1 for cheating). Using supervised learning, this project trained a combined approach of CNN-LSTM and XGBoost. The CNN-LSTM architecture focused on the sequences of moves, capturing patterns over time, while the XGBoost model made use of the summarized game data to improve the detection accuracy. Based on the combined predictions of both models, the expected output showed the final decision whether or not a player was flagged as a cheater.

2 Related work

This section discusses literature and empirical works relevant to this project proposal. Articles are gathered from sources such as Google Scholar, Academia, IEEE Xplore, ScienceDirect, DOAJ, Scopus, WorldCat, Springer, and ResearchGate. These referenced studies provide further background to make it easier to read and understand the purposes and primary elements of the research being presented.

Cheating, as defined by Consalvo [5], is the act of gaining an unfair advantage over opponents. On the other hand, an investigation by Yan & Randell revealed that cheating in chess includes "exploiting machine intelligence," where a cheating player gets an unfair advantage by receiving assistance from a strong chess computer/engine. It is important to have cheat detection in online chess because according to Barnes & Hernandez-Castro [1], from the work of Yan & Randell [30] focusing on the investigation of cheating in online games, just like other Internet games, chess has progressively become more susceptible to cheating via "machine intelligence". Its vulnerability is not due to the existence of chess engines per se, but because of how accessible chess engines are on ordinary personal computers, now far more powerful than any of even the best human players. Additionally, according to Zaksaitė [31] based from the article of McLaren titled "Corruption: Its Impact on Fair Play" [16], cheating at chess undermines the sport itself because it eliminates the unpredictability and integrity that make competition exciting. The game loses its status as a sport and becomes something more for entertainment purposes, where the focus is not on fairness in competition.

Barnes & Hernandez-Castro [1] state that cheating allegations have increased significantly in recent years which resulted to recent changes to the laws of the game, potentially breaking the players' privacy. A popular cheating allegation example is the case about chess Grandmaster Hans Niemann, who faced accusations of cheating through computer assistance, as stated by a chess.com report [4]. This report also explained the basics of cheat detection, such as: the chess clock time usage compared to the difficulty of the moves made, behavioral factors, comparing the moves made to the moves suggested by a computer, critical moves, etc. Likewise, Regan [22] discussed an approach using statistical analysis for detecting cheaters using methods like the Raw Outlier Index (ROI) and z-scores assessing player performance based on their skill level.

Defined by Merriam-Webster Dictionary [15], chess is a two-player game where each possesses 16 pieces and can move according to fixed rules across a 64-grid checkerboard where the goal is to checkmate each others' king. A widely used metric for determining skill level in chess is the "Elo rating system," which is developed by Arpad Elo and adopted by the World Chess Federation (FIDE) in 1970, as described by Berg [2]. Additionally, Hertl [9] states that the Portable Game Notation (PGN) is a widely used format for chess in order to record and share analysis of chess games online. Another common chess notation used is UCI, developed by Rudolf Huber and Stefan Meyer-Kahlen and released in November 2000, which is an open protocol that enables chess engines to communicate with other programs, including Graphical User Interfaces [28].

Nerkar et al. [20] defines chess engine as a computer program that analyzes chess positions and recommends the best moves. It processes the current chessboard setup to develop one or more moves that it evaluates to be most optimal according to its computations. The work of Menon [18] discussed the advancement of Artificial Intelligence (AI) showing how it is implemented for the creation of several chess engines including Stockfish.

The studies of both Menon [18] and Modi & Acuna [19] considered Stockfish as one of the best chess engines of all time. It was initially released in 2008 and it uses advanced Alpha-Beta search algorithms where it will rate positions depending on the mobility of pieces and the king safety. Stockfish uses the minimax algorithm where it assumes that each player in the chess game will find the most optimal move for their position. The engine will check moves and its respective outcomes while discarding less ideal moves with Alpha-Beta pruning to focus on more efficient ones. This system enables Stockfish to process an incredible number of positions in the shortest time possible, making it one of the most powerful tools in chess [18] [19] [25].

Maia is a chess engine created from a neural network which mimics a human-like style came from the study of McIlroy-Young et al. [17] [14] titled "Learning Models of Individual Behavior in Chess," which focused on creating AI systems that predict human behavior in chess. Its goal is to play the most "human" looking move, which is not necessarily the best move.

Ghosh et al. [8] define Convolutional Neural Network (CNN) as a special type of deep learning architecture that is inspired by the vision of living things. However, Shiri et al. [24] do not limit CNNs to only be applied for visual tasks but they can also be used in varying tasks including bioinformatics, speech recognition, object detection, and image classification. On the other hand, Shiri et al. [24] suggest that Long Short-Term Memory (LSTM) is an advanced type of Recurrent Neural Network (RNN) that can solve the issue of capturing long-term dependencies and was introduced by Hochreiter & Schmidhuber [10] in 1997. The study of Tasdelen & Sen [27], referenced from Donahue et al. [6], states that the CNN-LSTM model is the union of CNN layers that extract features from input data and LSTM layers providing prediction of sequences, which can be used for image labeling, image recognition, and labeling of videos. XGBoost was introduced by Chen & Guestrin [3], which is an open sourced scalable machine learning system used for tree boosting and gained popularity from competitions such as Kaggle, being used in many winning solutions.

Supervised learning, as defined by Liu & Wu [13], is a machine learning approach that aims to focus on the understanding of the relationship between the input and output using labeled training samples. These samples are known as supervised data, helping the system to learn to map inputs to outputs efficiently.

Several chess-related projects made use of CNNs and LSTMs. Nerkar et al. [20] focused on training a CNN to assess chess positions resembling to how chess grandmasters evaluate them, having the model measuring to have an accuracy of 67.23%. Meanwhile, Omori & Tadepalli [21] proposed a method estimating chess player ratings from clock times and game moves using a CNN-LSTM model achieving a Mean Absolute Error (MAE) of 182 rating points, showing a promise in improving chess rating systems. Beyond that, a Kaggle notebook created by Prajapati [23] demonstrated the use of the XGBoost algorithm along with the application of data cleaning and automated exploratory data analysis on a Kaggle chess cheating dataset [26], resulting in a performance of approximately 70% accuracy, which shows a decent level of predictive power.

This literature review displayed an exploration of cheat detection in online chess as well as the AI models and chess engines to be used. The strengths of this review include the application of machine learning models which assess player behavior. This project proposal aims to mend gaps in the field by combining CNNs, LSTMs, and XGBoost, as a novel contribution to help enhance the goal of identifying cheaters for intervention.

3 Dataset and Features

The dataset selected for this project is obtained from Kaggle Repositories [26], a platform for data scientists and machine learning engineers that allows users to find or publish datasets or collaborate with other individuals. It contains 48,000 chess games played between bots with each moves from these games having a chance of being generated by Stockfish (Cheat move) or Maia (Human move). The Maia bot has a fixed Elo rating for each game and is based from the first 10 moves of real human

players' games gathered from the Lichess open database [12]. For instance, if the initial 10 moves of a game feature one human player with an Elo rating of 1864 and the other with a rating of 1765, the Elo ratings for the Maia bot will be assigned as 1900 and 1800 respectively.

The Dataset

# Liste cheat white 0 for not cheating and 1 for cheating	# Liste cheat black 0 for not cheating and 1 for cheating	Game PGN format	# Elo White Elo for 10 first move white	# Elo Black Elo for 10 first move black	Score Result of the games
0 1.111111111111111e+162b	0 1.0111110101110111e+163b	46673 unique values	-1 2616	-1 2615	0-1 43% 1-0 40% Other (8323) 17%
00000000000011100111011111111111101101110111111	00000000000101110101000000010111000000100101100	1. d4 d5 2. c4 Nf6 3. Nc3 c6 4. Bg5 Qd6 5. c5 Qe6 6. Nf3 h6 7. Bh4 g5 8. Bg3 Nbd7 9. h3 Bg7 10. e3 N...	1201	1301	1/2-1/2
0000000000000101	0000000000100010	1. e4 e5 2. Bc4 Nf6 3. d3 Bc5 4. Nc3 c6 5. f4 d5 6. exd5 Nxd5 7. fxe5 Nxc3 8. bxc3 O-O 9. d4 Bb6 10....	1217	1301	1/2-1/2
00000000000111110110010111010101110111111	00000000000000000000000010000000000000000000	1. e4 e5 2. Nf3 Nf6 3. Nxe5 Qe7 4. Nf3 Nxe4 5. Bc4 Nc3+ 6. Kf1 Nxd1 7. Kg1 a6 8. d3 b5 9. Bb3 Nc6 10...	1228	1301	1/2-1/2
00000000000111010101000010011001011001110000100000000	00000000001111011111111111111111001111111	1. a3 e5 2. h3 Nc6 3. Nc3 Nf6 4. Nf3 d5 5. d3 e4 6. dxe4 dxe4 7. Qxd8+ Nxd8 8. Nd4 Bc5 9. e3 Bd7 10....	1246	1301	1/2-1/2

Figure 1: Sample data from the dataset

Moreover, this dataset completely includes a list of cheating indicators for both sides, where "Liste cheat white" means 0 for not cheating and 1 for cheating for the white side, and "Liste cheat black" has the same purpose for the black side. It also features each game in PGN format, results of the game ('1-0' as a win for white, '0-1' as a win for black, or '1/2-1/2' as a draw), and Elo ratings of the players referred to as "Elo White" and "Elo Black."

Combined Dataset

	Row	White Prediction	Black Prediction	Liste cheat white	Liste cheat black	Game	Elo White	Elo Black	Score	result
0	0	2013.659424	2015.224121	00000000001111101010111010100110000001001111	00000000001110010000000110010000000111000	1. d4 e6 2. c4 d5 3. Nc3 Be7 4. Nf3 Nf6 5. Bf4...	1001	-1	1/2-1/2	2
1	1	1996.151733	1997.726318	00000000000000101111011000001001110001111011111001	00000000001101111101111111111111111111011111111111	1. d4 c5 2. c4 cxd4 3. Qxd4 Nc6 4. Qd1 e6 5. N...	1001	-1	1/2-1/2	2
2	2	1511.751831	1513.163940	00000000001110011101111001111111101010111101111...	00000000001111111000011111111111111111111101010...	1. e4 e5 2. d4 exd4 3. Qxd4 Nc6 4. Qd3 b6 5. B...	1001	-1	1/2-1/2	2
3	3	1521.371582	1522.777832	00000000001111101111111111101101111111	00000000000001101001000111001111001110	1. e4 e5 2. Nf3 Nf6 3. Nc3 Bc5 4. Bc4 d6 5. O-...	1002	-1	1/2-1/2	2
4	4	1542.092529	1543.494873	00000000000100011101011110100101001101001011...	00000000000000001000001110011010100010001100...	1. d4 d5 2. c4 Nf6 3. Nc3 e6 4. Bg5 Be7 5. e3 ...	1002	-1	1/2-1/2	2
...	...	...	...	...	...	...	...	...	...	...
48927	48927	1550.246948	1551.718384	00000000000000000000000000000000	00000000001001000000110010111	1. e4 e5 2. Qh5 Qe7 3. Bc4 Nf6 4. Qf3 d6 5. h3...	1841	1806	0-1	1
48928	48928	1337.956055	1339.301147	00000000000000000000000000000000000000000000	00000000001100110111100111111101110101001	1. c4 d5 2. c5 c6 3. b4 Bf5 4. d3 e6 5. Bf4 Nd...	1562	1561	0-1	1
48929	48929	1765.669678	1767.174927	00000000000000000000000000000000000000000000000...	00000000001001101001000001000010001001001010...	1. e4 e5 2. Bc4 Nf6 3. Nf3 d6 4. Ng5 Be6 5. Bx...	1476	1420	1-0	0
48930	48930	1700.737915	1702.206543	00000000000000000000000000000000	00000000001000001000000010100	1. d4 Nf6 2. c4 g6 3. Nc3 e6 4. Nf3 Bg7 5. e4 ...	1745	1536	1-0	0
48931	48931	1950.847778	1952.405396	00000000000000000000000000000000000000000000000...	00000000000000000100100000000000001000100000...	1. e4 e5 2. Nc3 d6 3. h3 Be6 4. Nf3 Nf6 5. d4 ...	1656	1613	1-0	0

48932 rows × 10 columns

Figure 2: Sample data from the dataset combined with the CNN-LSTM model predictions

The predictions generated by the pre-trained CNN-LSTM model from the academic paper of Omori Tadepalli [21] provided estimated Elo ratings for the players which are named "White Prediction" for the white player and "Black Prediction" for the black player. These predicted ratings were added to

the main dataset as additional features. The inclusion of these estimated Elo ratings, alongside the original game data, allowed for a more comprehensive analysis and were used to train the XGBoost classifier to enhance the accuracy of detecting cheating behaviors in online chess games.

4 Methods

This section shows the procedure of how the CNN-LSTM and the XGBoost algorithm was implemented for training the chess anti-cheat program to detect suspected cheaters. Based on move sequences and compiled game data, the trained AI will examine chess games to find trends of cheating behavior. A supervised learning method was employed in training the AI to identify players who are cheating and not cheating. Machine learning platforms such as PyTorch and Scikit-learn along with other Python libraries were used to train the models using supervised learning focusing on capturing patterns from the sequences of moves and combining each.

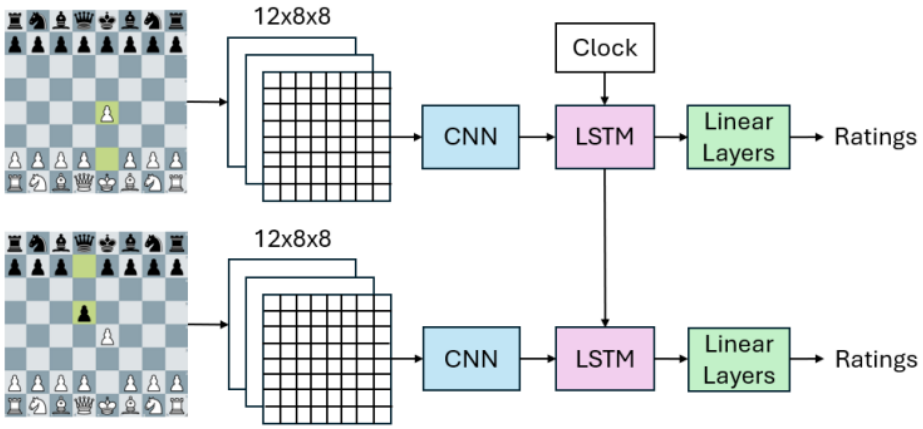


Figure 3: Diagram of the CNN-LSTM architecture adapted from Omori & Tadepalli (2024) showing the first two time steps of a sample game.

CNN-LSTM is a hybrid framework integrating Convolutional Neural Networks (CNN) combined with Long Short-Term Memory (LSTM) networks designed to extract features from input data and make predictions based on sequences [6] [27]. The CNN-LSTM model by Omori Tadepalli [21], which comprises a CNN to learn positional features, integrates these features with clock-time data into a Bidirectional LSTM to predict player ratings after each move. The model, trained with 80% of the data for training and 20% for testing, achieved a mean absolute error (MAE) of 183 rating points on the test set, indicating strong performance.

XGBoost is an open-source machine learning framework designed to assist computers in making accurate predictions based on data [3]. In this project, XGBoost is trained on the combined predicted player ratings and the original dataset, such as player Elo ratings, game outcomes, and cheating indicators.

Supervised learning is a type of machine learning method where a computer is tasked to learn from examples that come with labels [13]. Both the CNN-LSTM and XGBoost models were trained using supervised learning techniques on labeled data, including cheating status, Elo ratings, and game results.

The dataset first underwent preprocessing, where each Portable Game Notation (PGN) move sequence is converted into corresponding board states. To achieve this, a modified version of code from the previously mentioned related paper on chess Elo rating prediction using CNN-LSTM was utilized [21], with permission from the authors. A Python script was developed to automate this process, parsing game data into individual board states. Each move's board state is saved as a pickle (.pkl) file in a designated directory called "games," creating a comprehensive representation of the games in a machine-readable format.

```
WhiteElo,BlackElo,Result,UCIMoves,Positions,Clocks
1001,-1,1/2-1/2,d2d4 e7e6 c2c4 d7d5 b1c3 f8e7 g1f3 g8f6 c1f4 e8g8
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [1., 1., 1., 1., 1., 1., 1., 1.],
         [0., 0., 0., 0., 0., 0., 0., 0.]],

        [[0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 0., 0., 0., 0., 0., 0., 0.],
         [0., 1., 0., 0., 0., 0., 1., 0.]],

        [[0., 0., 0., 0., 0., 0., 0., 0.],
```

Figure 4: Preprocessed .csv game data with board States converted from UCI, which was also converted from the algebraic chess notation in found in chess PGN.

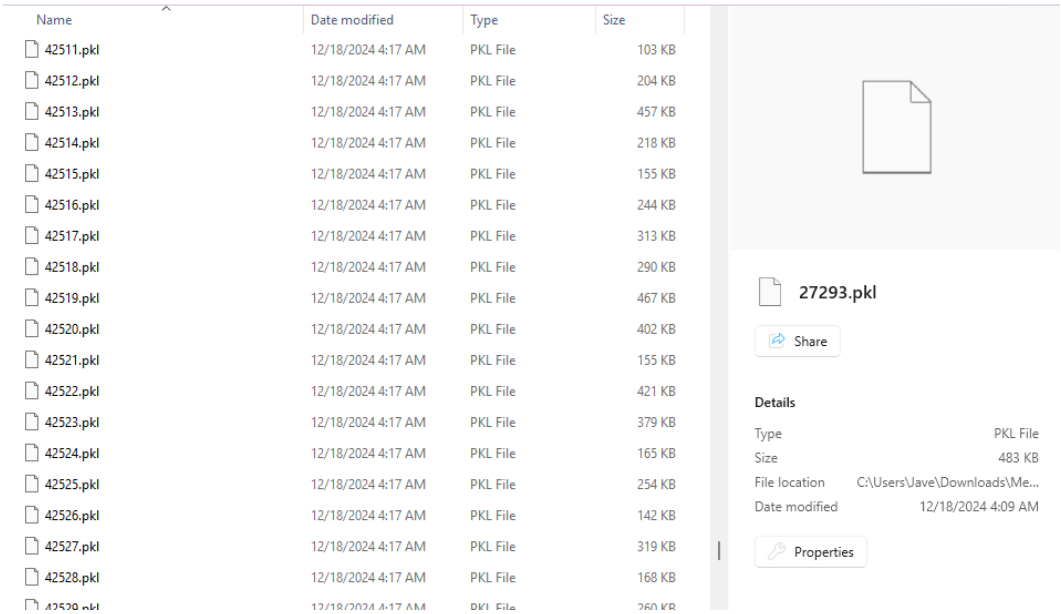


Figure 5: Processed pickle (.pkl) files of each individual chess game taking up 13 Gigabytes of storage.

One of the key features for detecting suspicious behavior in this project is the predicted Elo ratings of players. These predictions were generated using a pre-trained CNN-LSTM model referenced in the paper by Omori & Tadepalli [21]. The model was applied to each game in the dataset through a script designed to estimate player ratings. The resulting predictions, referred to as "White Prediction" and "Black Prediction," were then incorporated into the original dataset. A custom script was developed to automate this process, ensuring that the predicted Elo ratings for both players were seamlessly incorporated as new features into the main dataset. This combined dataset now contains a combination of critical features such as player Elo ratings, game outcomes, and the predicted ratings. This dataset served as the input for training the XGBoost classifier, which was used to analyze the data and produce the final results.

5 Experiments/Results/Discussion

In this section, we analyze the performance of the classification model using both quantitative metrics and qualitative visualizations. The evaluation highlights the strengths and weaknesses of the model, providing insights into its behavior on the given dataset.

The primary metrics used to assess the model are **accuracy**, **precision**, **recall**, and **F1-score**. The confusion matrix further illustrates the distribution of correct and misclassified predictions across the classes, while a detailed correlation heatmap examines relationships between features.

The hyperparameters were selected through a trial-and-error process with the goal of optimizing the model's performance. The values were adjusted based on validation accuracy, aiming to balance model complexity, training speed, and overfitting prevention. The chosen values, such as **max_depth: 7**, **learning_rate: 0.05**, **colsample_bytree: 1.0**, **subsample: 0.7**, **reg_alpha: 1**, **reg_lambda: 1**, **n_estimators: 100**, and **random_state: 42**, were determined iteratively to achieve the best trade-off between model performance and computational efficiency.

5.1 Quantitative Results

Confusion Matrix

The confusion matrix, presented below, summarizes the prediction results of the model:

$$\begin{bmatrix} 3258 & 620 & 27 \\ 956 & 3213 & 36 \\ 912 & 526 & 239 \end{bmatrix}$$

The confusion matrix reveals the following trends:

- **Class 0** (majority class) achieves the highest recall, with 83% of true instances correctly identified. However, there are still significant false negatives where instances of Class 0 are misclassified into Class 1.
- **Class 1** demonstrates balanced performance, achieving a precision of 74% and a recall of 76%. This reflects the model's ability to both identify and correctly predict this class.
- **Class 2** performs the weakest, with a recall of only 14%, indicating that the model struggles to identify true positives for this class. A large portion of Class 2 instances are incorrectly predicted as Class 0 or Class 1.

Classification Metrics

The performance metrics, computed across all three classes, are summarized in the table below:

Metric	Class 0	Class 1	Class 2	Overall
Precision	0.64	0.74	0.79	–
Recall	0.83	0.76	0.14	–
F1-Score	0.72	0.75	0.24	–
Support	3905	4205	1677	9787
Accuracy	0.69			
Validation Accuracy	0.6824			

Precision measures how accurately the model predicts positive instances relative to all predicted positives. The model achieves high **precision** for Class 2, meaning when it predicts Class 2, it is generally correct. **Recall**, on the other hand, measures how many true positive instances the model identifies. Although the model performs well for Classes 0 and 1 in terms of recall, the recall for Class 2 remains low. This reflects the model's difficulty in correctly identifying instances from the minority Class 2. The **F1-score**, which balances precision and recall, shows balanced performance for Classes 0 and 1, but highlights underperformance for Class 2.

The overall accuracy of the model on the test set is 69%, while the validation accuracy converges to 68.24%. These metrics suggest that the model generalizes reasonably well across unseen data, though opportunities for improvement remain.

5.2 Qualitative Results

Confusion Matrix Analysis

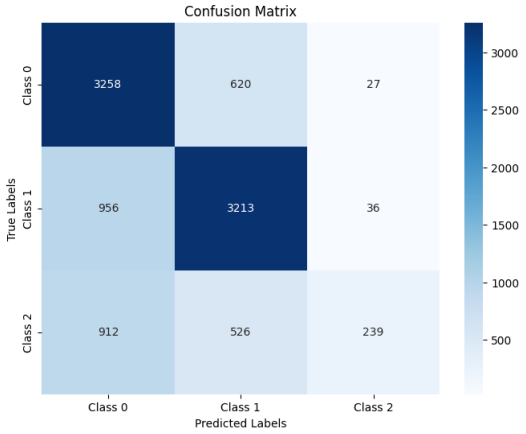


Figure 6: The confusion matrix above provides a detailed breakdown of the model's predictions across the three classes. Each row represents the actual class, while each column represents the predicted class.

The diagonal elements (3258, 3213, and 239) indicate correctly classified samples for Class 0, Class 1, and Class 2, respectively. Off-diagonal elements represent misclassifications. Specifically, 620 samples from Class 0 were misclassified as Class 1, and 27 were misclassified as Class 2. For Class 1, 956 samples were predicted as Class 0, and 36 were predicted as Class 2. Similarly, for Class 2, 912 samples were incorrectly predicted as Class 0, and 526 as Class 1.

The overall accuracy of the model is 0.69, as previously stated. However, there is significant performance disparity for Class 2, which has the lowest recall (0.14). This highlights the model's difficulty in correctly identifying samples of Class 2 compared to Classes 0 and 1.

Feature Correlation Analysis

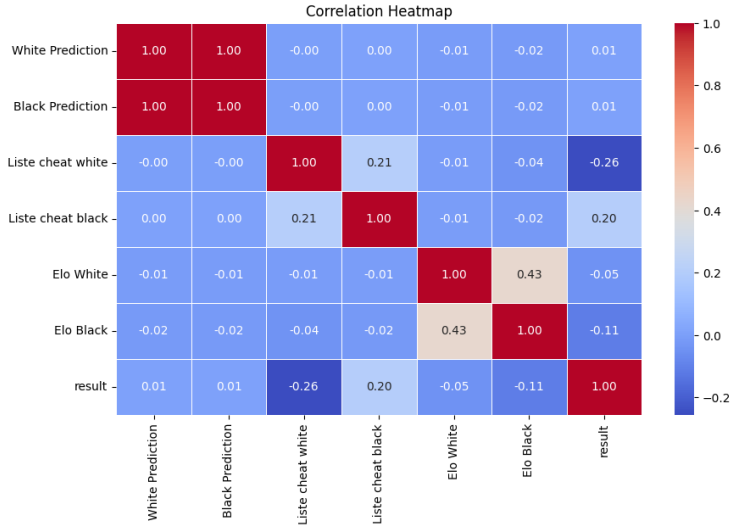


Figure 7: The correlation heatmap provides a visualization for the relationships between the features used in the model.

Strong correlations ($r = 1.00$) are observed between *White Prediction* and *Black Prediction*, suggesting that these features may provide similar or redundant information to the model. A moderate correlation ($r = 0.43$) is found between *Elo White* and *Elo Black*, which aligns with the expectation that these values reflect the competitive balance between players. Interestingly, the *result* variable shows weak or even negative correlations with most features, indicating that the outcome may not be easily predicted based solely on the available features.

5.3 Discussion of Results

The model achieves an overall accuracy of 69%, showing good performance on Classes 0 and 1. However, it struggles significantly with Class 2, where the recall is only 14%. This issue is largely due to class imbalance in the dataset, where Class 2 has far fewer examples (1677 instances) compared to the other two classes.

The confusion matrix confirms that instances of Class 2 are often misclassified as Class 0 or Class 1. These misclassifications likely occur because the decision boundaries between these classes are less clear, especially due to the class imbalance.

The correlation heatmap reveals that some input features are highly interdependent, which may reduce the model's ability to make accurate predictions. In future iterations, feature engineering or dimensionality reduction techniques could help address these redundancies and improve the model's performance.

The confusion matrix confirms that instances of Class 2 are often misclassified into Class 0 or 1. Such misclassifications likely arise because the decision boundary between these classes is less distinct, exacerbated by the imbalance.

Additionally, the correlation heatmap suggests that some input features may be highly interdependent, potentially limiting the model's ability to generalize. Feature engineering or dimensionality reduction techniques could address these redundancies in future iterations.

6 Conclusion/Future Work

In summary, the model achieved an overall accuracy of 69% and a validation accuracy of 68.24%. The XGBoost algorithm performed the best, as it effectively handled the imbalance in the dataset and leveraged tree-based methods that reduced overfitting. The combination of CNN-LSTM and XGBoost helped capture both the patterns in chess moves and the overall game data, contributing to improved detection accuracy. However, the model struggled with Class 2 due to class imbalance and unclear decision boundaries between classes.

If more time, team members, or computational resources were available, future work could focus on addressing class imbalance through techniques like oversampling or using more advanced models such as generative models. Exploring other modeling approaches, such as ensemble methods or more complex neural networks, could also improve performance, especially for Class 2. Additionally, refining feature selection and reducing feature dependencies using dimensionality reduction techniques could help the model generalize better.

Overall, despite these challenges, the current model lays a solid foundation for further improvements and research into cheat detection systems for online chess platforms

References

[1] Barnes, D. J., & Hernandez-Castro, J. (2014). On the limits of engine analysis for cheating detection in chess. *Computers & Security, 48*, 58–73. https://doi.org/10.1016/j.cose.2014.10.002

[2] Berg, A. (2020). Statistical analysis of the ELO rating system in chess. *CHANCE, 33*(3), 31–38. https://doi.org/10.1080/09332480.2020.1820249

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. Presented at the San Francisco, California, USA. doi:10.1145/2939672.2939785

[4] Chess.com Team (CHESScom). (2022, October 5). Hans Niemann report. Chess.com. https://www.chess.com/blog/CHESScom/hans-niemann-report

[5] Consalvo, M. (2008, January 1). Cheating: gaining advantage in videogames. Mia Consalvo - Academia.edu. https://www.academia.edu/79675726/Cheating_gaining_advantage_in_videogames

[6] Donahue, J., Hendricks, L. A., Rohrbach, M., Venugopalan, S., Guadarrama, S., Saenko, K., & Darrell, T. (2014, November 17). Long-term recurrent convolutional networks for visual recognition and description. *arXiv.org*. https://arxiv.org/abs/1411.4389

[7] Duca Iliescu D. M. (2020). The Impact of Artificial Intelligence on the Chess World. *JMIR Serious Games, 8*(4), e24049. https://doi.org/10.2196/24049

[8] Ghosh, A., Sufian, A., Sultana, F., Chakrabarti, A., & De, D. (2019). Fundamental concepts of Convolutional Neural Network. In *Intelligent systems reference library* (pp. 519–567). https://doi.org/10.1007/978-3-030-32644-9_36

[9] Hertl, B. V. (2022). Explanatory analysis of the chess game.

[10] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[11] Iliescu, D. M. D. (2020). The impact of artificial intelligence on the chess world. *JMIR Serious Games, 8*(4), e24049. https://doi.org/10.2196/24049

[12] lichess.org open database. (n.d.). https://database.lichess.org/

[13] Liu, Q., & Wu, Y. (2012). Supervised learning. In *Springer eBooks* (pp. 3243–3245). https://doi.org/10.1007/978-1-4419-1428-6_451

[14] Maia Chess. (n.d.). https://www.maiachess.com/

[15] Merriam-Webster. (n.d.). Chess. In *Merriam-Webster.com dictionary*. Retrieved October 18, 2024, from https://www.merriam-webster.com/dictionary/chess

[16] McLaren, R. H. (2008). Corruption: Its impact on fair play. *Marq. Sports L. Rev., 19*, 15.

[17] McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J., & Anderson, A. (2022). Learning models of individual behavior in chess. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. https://doi.org/10.1145/3534678.3539367

[18] Menon, A. (2022). The impact of Artificial Intelligence (AI) and Engines on Boardgames (Chess and Go). *International Journal of High School Research, 4*(2), 65–70. https://doi.org/10.36838/v4i2.12

[19] Modi, E., & Acuna, K. (2023). The effects of computer and AI engines on competitive chess. *Journal of Student Research, 12*(3). https://doi.org/10.47611/jsrhs.v12i3.4993

[20] Nerkar, A., Nambiar, Y., Shrotri, N., & Bhavsar, S. S. (2022). Issue 2 www.ijrar.org (E-ISSN. ResearchGate.

[21] Omori, M., & Tadepalli, P. (2024). Chess Rating Estimation from Moves and Clock Times Using a CNN-LSTM. *ArXiv, abs/2409.11506*.

[22] Regan, K. (n.d.). Fraught issues in statistical chess cheating detection. Retrieved from https://www.cs.buffalo.edu/ regan/

[23] Prajapati, R. (2024, April 12). Data cleaning + Auto-EDA + XgBoost (70

[24] Shiri, F. M., Perumal, T., Mustapha, N., & Mohamed, R. (2023, May 27). A comprehensive overview and comparative analysis on deep learning models: CNN, RNN, LSTM, GRU. *arXiv.org*. https://arxiv.org/abs/2305.17473

[25] Stockfish Developers. (n.d.). Stockfish chess engine. *Stockfish*. https://stockfishchess.org/

[26] Spotting Cheaters: Chess cheating dataset. (2024, April 1). Kaggle. https://www.kaggle.com/datasets/brieucdandoy/chess-cheating-dataset/data

[27] Tasdelen, A., & Sen, B. (2021). A hybrid CNN-LSTM model for pre-miRNA classification. *Scientific Reports, 11*(1). https://doi.org/10.1038/s41598-021-93656-0

[28] UCI - Chessprogramming wiki. (n.d.). https://www.chessprogramming.org/UCI

[29] Weber, B. (1997, May 12). Swift and slashing, computer topples Kasparov. *The New York Times*. https://www.nytimes.com/1997/05/12/nyregion/swift-and-slashing-computer-topples-kasparov.html

[30] Yan, Jeff & Randell, Brian. (2009). An Investigation of Cheating in Online Games. *IEEE Security & Privacy*, 7, 37-44. 10.1109/MSP.2009.60.

[31] Zaksaitė, S. (2020). Cheating in chess: a call for an integrated disciplinary regulation. *Kriminologijos Studijos, 8*, 57–83. https://doi.org/10.15388/crimlithuan.2020.8.3